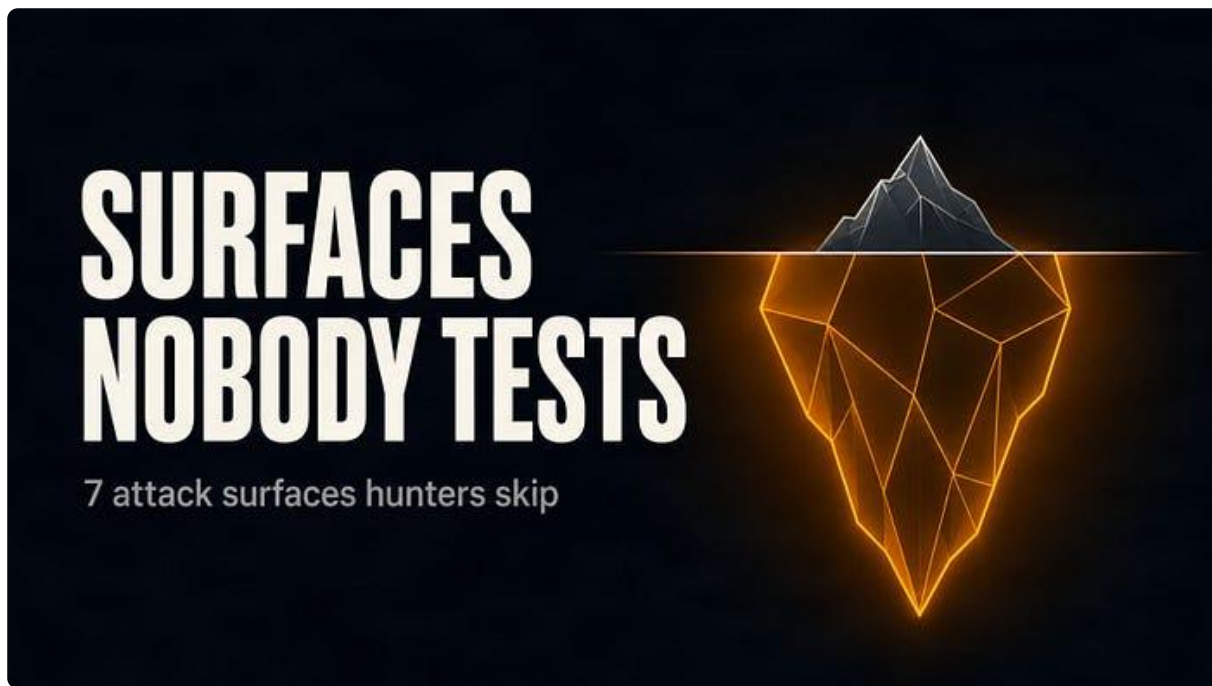




Most Hunters Test the Same Surface as Everyone Else.

The homepage has been tested by 500 hunters. The attack surface nobody is looking at has been tested by zero. Here is how to find it.

Abhishek meena

Published August 3, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

Most Hunters Test the Same Surface as Everyone Else. Here Is How to Find Surfaces Nobody Is Looking At.	3
1. Non-standard ports	4
2. Staging and development environments	4
3. JavaScript files	5
4. Mobile apps	6
5. Acquired companies	7
6. CI/CD and internal tooling	7
7. The changelog as an attack surface map	7
The pattern	8
Sources	8

Most Hunters Test the Same Surface as Everyone Else. Here Is How to Find Surfaces Nobody Is Looking At.

30-second version

Most hunters run the same recon tools against the same domains on the same ports and wonder why they get duplicates. The bugs are not on the homepage. They are on the surfaces nobody else is checking: non-standard ports, staging environments, JavaScript files, mobile apps, acquired companies, and internal tooling. Here are 7 surfaces that consistently produce bugs because hunters skip them. Every one of them is findable with free tools and a change in where you point them.

I spent my first year in bug bounty doing what every tutorial told me to do.

Run `subfinder` on the main domain. Run `nuclei` on every subdomain that responded on port 443. Run `ffuf` for directory discovery. File reports. Get duplicates.

The pattern was always the same. I was testing the same surface as every other hunter who read the same blog posts and watched the same YouTube videos. We were all standing in the same field, swinging metal detectors at the same patch of ground.

The turn came when I started studying how top researchers actually find bugs. Not the talks. Not the tool demos. The actual methodology behind **where** they point their tools.

Shubs, ranked top 30 on HackerOne globally, put it plainly: he spent 40+ hours just understanding a single company's attack surface before testing anything. Not running tools. Building a mental picture of how the company deploys code, what technologies they use, and where their assets live.

NahamSec found a critical SSRF on Lyft not by testing the main app, but by exploring a feature nobody else looked at: the expense report PDF generator. He discovered the underlying library (WeasyPrint) through the User-Agent, downloaded the source code, built a local replica, and found the exploit path. Nobody else was even looking at that feature.

The difference is not skill. The difference is **where you look**.

Here are 7 surfaces that consistently produce bugs because most hunters skip them.

1. Non-standard ports

Most hunters check port 80 and port 443. That is it.

The application running on port 8443, 9090, or 3000 is often a different service entirely. Internal APIs, admin panels, development servers, Grafana dashboards, Jenkins instances, and debug endpoints live on non-standard ports. They are deployed by developers who needed a quick way to expose something internally and forgot to restrict access.

```
# Don't just scan 80 and 443. Scan everything.
nmap -sS -p- -T4 --min-rate 5000 <target-ip>

# Or faster, masscan for discovery then nmap for service detection
masscan -p1-65535 <target-ip> --rate=10000
```

I have found more bugs on port 8443 and port 9090 than on port 443. Not because those ports are more vulnerable. Because nobody else is looking there.

2. Staging and development environments

Production is hardened. Staging is not.

Staging environments are where developers test changes before shipping. They often have weaker authentication, debug mode enabled, verbose error messages, and test accounts with predictable credentials. They run the same code as production but with fewer security guardrails.

Finding them:

- **Certificate Transparency logs** at `crt.sh` for patterns like `staging.`, `dev.`, `qa.`, `test.`, `uat.`, `sandbox.`, `preprod.`
- **DNS records** that reveal internal naming conventions (`internal-api.`, `admin.`, `backend.`)
- **AltDNS** to generate permutations of known subdomains and resolve them

```
# Find staging patterns in CT logs
curl -s "https://crt.sh/?q=%25.staging%.target.com&output=json" | jq -r '[][.name_value]' |
sort -u
# Generate and resolve permutations
altdns -i subdomains.txt -o alt_subdomains.txt -r -s -t 5
massdns -r resolvers.txt -t A alt_subdomains.txt
```

A staging environment often has the same API as production but without rate limiting, without WAF, and with debug endpoints enabled. That is where the bugs are.

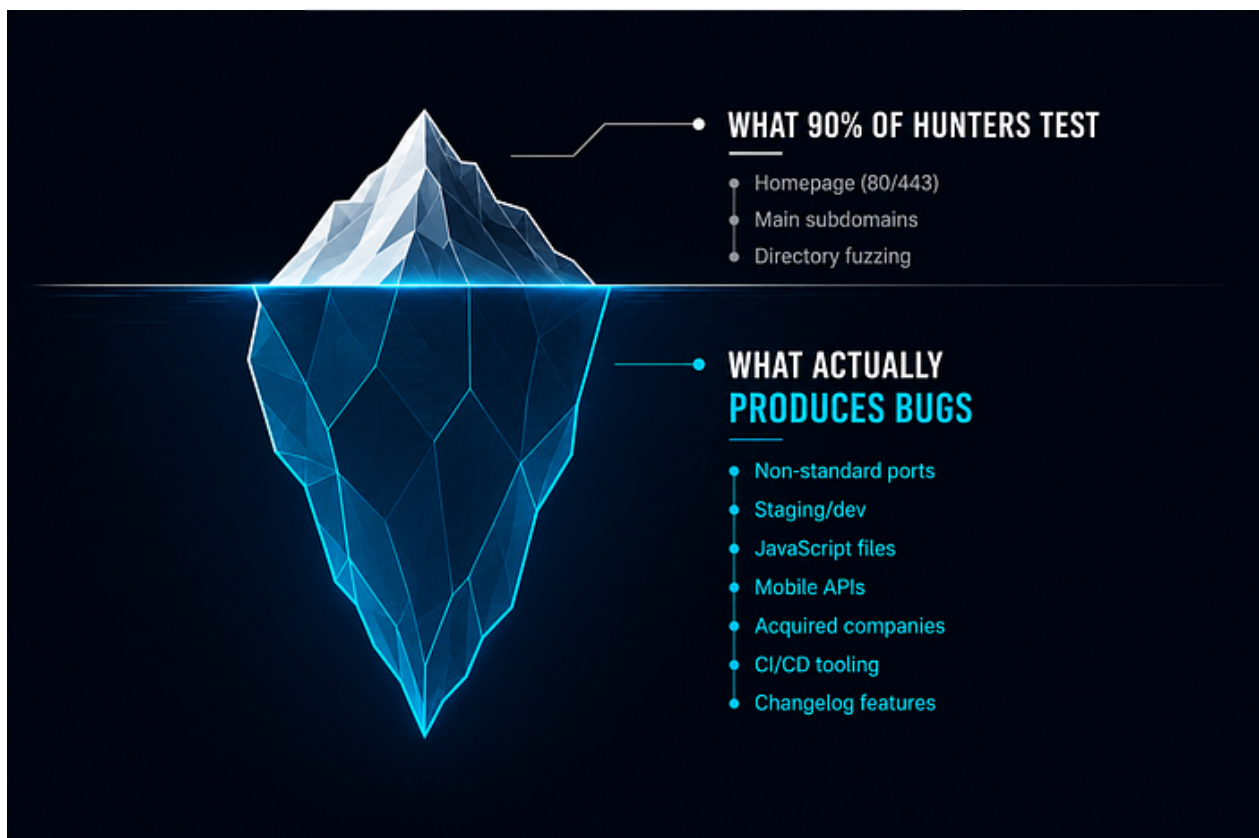
3. JavaScript files

Every modern web app loads JavaScript files. Those files contain endpoints, API keys, internal URLs, feature flags, and configuration data that the application does not expose in its UI.

Most hunters load the page in Burp and look at the requests the app makes. They miss the endpoints that are referenced in JavaScript but never called during normal usage: deprecated features, admin routes, beta endpoints, debug toggles.

```
# Extract all JS files from a target
katana -u https://target.com -jc -o js_files.txt
# Extract endpoints from JS files
cat js_files.txt | while read url; do
    curl -s "$url"
done | grep -oE '(https?://|/)[a-zA-Z0-9_./?=&%-]+' | sort -u
```

The endpoints hiding in JavaScript files are the ones the developers forgot to remove. They are the admin panels, the deprecated APIs, and the debug routes that still work. Nobody else is reading the source code of the JavaScript. That is why it pays.



4. Mobile apps

Most hunters never look at the mobile app. They test the web application and stop.

The mobile app often talks to a different API than the web app. That API has different endpoints, different authentication, different rate limiting, and different access control. It is built by a different team, deployed on different infrastructure, and tested by nobody in the bug bounty programme.

Download the APK. Decompile it. Extract the API endpoints.

```
# Download the APK
apkleak -p com.target.app -o target.apk
# Decompile
jadx -d target_decompiled target.apk
# Find API endpoints in the decompiled code
grep -rn "https\?:://" target_decompiled/ | grep -v "schemas.android" | sort -u
```

Inon Shkedy's API security tip #30: download old versions of the APK from APKPure. Companies do not always implement security mechanisms from day one, and DevOps engineers do not often deprecate old APIs. The old APK exposes endpoints that have no authentication, no rate limiting, and no authorisation checks.

5. Acquired companies

When a company acquires another, their attack surface expands overnight. But the bug bounty scope often does not update immediately.

The acquired company's infrastructure, APIs, and subdomains are now part of the parent company's network. They may share authentication systems, have cross-domain trust relationships, or expose internal APIs that the parent company's infrastructure trusts.

Check the parent company's acquisitions page, Crunchbase, and press releases. Find the acquired company's domains. Check if they are in scope. Test them.

Acquired company assets are often running older code, with fewer security updates, and nobody on the parent company's security team has audited them yet. That is the definition of an under-tested surface.

6. CI/CD and internal tooling

Jenkins, GitLab, Grafana, Kibana, Confluence, Jira, and internal dashboards are exposed to the internet more often than companies admit.

These tools run on non-standard ports, are deployed by DevOps teams (not security teams), and often have default credentials, unauthenticated endpoints, or known CVEs that were never patched because nobody is responsible for updating them.

```
# Check for exposed CI/CD tools
nuclei -tExposedPanel/ -u https://target.com
nuclei -tExposedFiles/ -u https://target.com
# Common paths to check
/jenkins/script
/.git/config
/actuator/health
/actuator/env
/healthz
/debug/pprof/
/server-status
```

Orange Tsai's research on Jenkins showed how unauthenticated RCE was possible through dynamic routing. The technique still works because Jenkins instances are deployed by developers, not secured by security teams, and exposed to the internet by accident.

7. The changelog as an attack surface map

This one is not a surface. It is a method for finding surfaces.

Every time a company ships a new feature, a new API, or a new integration, they create a new attack surface. That surface has been tested by zero hunters because it did not exist yesterday.

Check:

- **Release notes** on the company blog
- **App store updates** ("what's new" sections)
- **GitHub commits** if the target has a public repository
- **Product roadmap pages** that describe upcoming features

The feature that shipped last Tuesday is the least tested code in the entire programme. The homepage has been tested by 500 hunters. The feature that shipped last Tuesday has been tested by none.

The pattern

Look at all 7 surfaces. The pattern is the same in every one.

The bugs are not where everyone is looking. They are where nobody is looking.

- Non-standard ports: nobody scans past 443
- Staging environments: nobody checks CT logs for dev patterns
- JavaScript files: nobody reads the source code
- Mobile apps: nobody downloads the APK
- Acquired companies: nobody checks what the parent company bought
- Internal tooling: nobody scans for Jenkins on port 8080
- Changelogs: nobody reads release notes

Every one of these is findable with free tools. The cost is not tooling. The cost is the habit of pointing your tools where everyone else is pointing theirs.

Shubs said it best: spending 40+ hours understanding a single company's attack surface before testing anything is what separates consistent earners from hunters who get duplicates. The tools are the same. The surface is different.

Stop testing the homepage. Start testing everything below the waterline.

Sources

- Shubs (Assetnote), "So, you want to get into bug bounties?" — shubs.io/so-you-want-to-get-into-bounties/

- NahamSec, "My Expense Report resulted in a Server-Side Request Forgery (SSRF) on Lyft" — nahamsec.com/posts/my-expense-report-resulted-in-a-server-side-request-forgery-ssrf-on-lyft
- Assetnote, "A Glossary of Blind SSRF Chains" — blog.assetnote.io/2021/01/13/blind-ssrf-chains/
- Inon Shkedy, "31 Days of API Security Tips" — github.com/inonshk/31-days-of-API-Security-Tips
- Orange Tsai, "Hacking Jenkins Part 1" — blog.orange.tw/2019/01/hacking-jenkins-part-1-play-with-dynamic-routing.html